Université de Caen Normandie
UFR des Sciences
Licence 2 Informatique
Projet 1

Rapport de Projet

Interpréteur de systèmes de Lindenmeyer (L-système) en Java

ACAR Thomas
GHALMI Zineddine
LETE Arthur

18 avril 2026

Résumé

Cette étude porte sur la modélisation et la visualisation de structures fractales à travers les systèmes de Lindenmayer (L-systèmes). L'objectif est de traduire des règles de production formelles en motifs géométriques complexes par le biais d'algorithmes optimisés, en utilisant Java. Notre système prend en charge une large gamme de modèles, allant des systèmes déterministes classiques (D0L-systèmes) aux approches stochastiques et contextuelles. L'architecture logicielle intègre nativement un moteur de rendu en deux dimensions (2D) et en trois dimensions (3D), permettant une modélisation spatiale fidèle des processus de croissance biologique. Les résultats obtenus démontrent l'efficacité de ces langages formels pour générer des géométries fractales sophistiquées tout en garantissant une performance fluide.

Table des matières

1 Introduction **2**
- 1.1 Description générale . . . 2
- 1.2 Identification de la problématique . . . 2
- 1.3 Présentation du plan . . . 3

2 Structuration du projet **5**
- 2.1 Description des besoins . . . 5
  - 2.1.1 Configuration par fichier externe et parsing . . . 5
  - 2.1.2 Interprétation graphique et moteur de rendu . . . 5
- 2.2 Fonctionnalités implémentées . . . 6
- 2.3 Organisation du projet . . . 7

3 Éléments techniques **8**
- 3.1 Description des structures de données . . . 8
  - 3.1.1 Gestion des Symboles via le pattern Flyweight : . . . 8
  - 3.1.2 Encapsulation des Règles de Production : . . . 8
  - 3.1.3 Mappage des Symboles aux Actions : . . . 9
  - 3.1.4 L'état de `Turtle2D` . . . 9
- 3.2 Description des algorithmes . . . 9
  - 3.2.1 Algorithme de sélection stochastique (Tirage aléatoire) : . . . 10
  - 3.2.2 Algorithme de développement récursif (DFS) : . . . 10
  - 3.2.3 Algorithme de développement itératif avec Singletons (BFS) : . . . 12

4 Architecture du projet **13**
- 4.1 Diagrammes des modules et des classes . . . 13
  - 4.1.1 Structure du paquetage `turtle` . . . 13
  - 4.1.2 Structure du paquetage `fractal` . . . 14
- 4.2 Description des paquetages utilisés . . . 14
- 4.3 Chaînes de traitement . . . 14

5 Expérimentations et usages **17**
- 5.1 Cas d'utilisation . . . 17
  - 5.1.1 Grammaire du fichier de configuration . . . 17
  - 5.1.2 Algorithme de compilation / execution automatisé (ANT) . . . 19
  - 5.1.3 Aperçu des résultats visuels . . . 20
- 5.2 Résultats quantifiables . . . 20

6 Conclusion **23**

Chapitre 1

Introduction

1.1 Description générale

[1] Un système de Lindenmayer ou L-systèm est un système de récriture permettant de générer des chaînes de caractères à partir de règles. Il est inspiré par la biologie et inventé en 1968 par le biologiste hongrois Aristid Lindenmayer. Un L-système a deux composantes principales :

1. une chaîne de départ,
2. au moins une règle de récriture.

Pour illustrer ce concept, considérons la courbe de Koch, définie par :

— **Axiome (chaîne de départ) :** F
— **Règle de production :** $F \to F - F + +F - F$

Dans ce formalisme, chaque caractère possède une interprétation graphique spécifique. Par exemple, le caractère **F** ordonne de tracer un segment, tandis que + et **-** correspondent à des rotations d'un angle fixe (ici 60°).

En appliquant la règle de réécriture, nous passons d'un simple segment à des structures de plus en plus complexes :

— **Évolution 0 :** F (un segment droit)
— **Évolution 1 :** $F - F + +F - F$ (une forme de "pointe". Figure 1.1)
— **Évolution 2 :** $F-F++F-F-F-F++F-F++F-F++F-F-F-F++F-F$
Chaque F de l'étape précédente est à nouveau remplacé, générant une chaîne encore plus longue qui, une fois interprétée, forme une fractale détaillée.

Cette approche itérative permet de modéliser des structures auto-similaires complexes à partir de règles extrêmement simples.

1.2 Identification de la problématique

L'un des défis majeurs de ce projet réside dans la gestion efficace de la mémoire lors de la génération des chaînes de caractères complexes. Pour produire un L-système, la méthode la plus intuitive consiste à stocker l'axiome sous forme de **String** et à remplacer chaque caractère par sa règle de production correspondante à chaque nouvelle itération.

FIGURE 1.1 – Courbe de Koch - Évolution 1

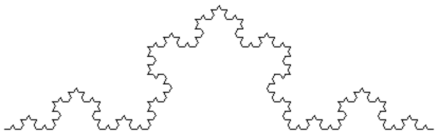

FIGURE 1.2 – Courbe de Koch - Évolution 4

Cependant, en **Java**, cette approche naïve est extrêmement inefficace. Étant donné que les nouvelles générations peuvent être construites de manière itérative (ou parallèle), nous devons conserver l'ancienne chaîne tout en construisant la nouvelle. En Java, les objets String sont **immuables** : chaque concaténation d'un caractère à une chaîne de longueur n entraîne la création d'une nouvelle instance et la copie de l'intégralité du contenu existant. [2] [3]

Le nombre total d'opérations de copie est donc défini par la somme des entiers :

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

Par conséquent, la construction d'une génération de longueur n par ajouts successifs présente les complexités suivantes :

— **Complexité en temps :** $O(n^2)$, car chaque ajout coûte $O(k)$ où k est la longueur actuelle.
— **Complexité en espace :** $O(n^2)$ en raison de la création massive d'objets temporaires avant d'atteindre la chaîne finale.

L'enjeu de ce projet est donc de s'affranchir de cette approche naïve en implémentant une interprétation optimisée. Cette optimisation nous permettra d'atteindre un nombre d'itérations bien plus élevé, impossible à obtenir avec une gestion classique des chaînes de caractères.

1.3 Présentation du plan

Ce rapport est structuré de manière à retracer le cycle complet de conception et de réalisation de l'interpréteur. Dans un premier temps, nous exposerons les objectifs techniques fixés ainsi que l'organisation générale du projet.

Ensuite, nous détaillerons les choix critiques concernant les structures de données et les algorithmes optimisés. Cette partie explique de manière simplifiée pourquoi certains choix techniques ont été cruciaux pour garantir la performance du programme.

Par la suite, nous présenterons l'architecture globale du logiciel, en mettant en avant l'interaction entre les différents modules. Enfin, nous terminerons par une phase d'expérimentations et d'usages, où nous analyserons les résultats obtenus à travers diverses démonstrations et cas d'utilisation concrets en 2D et 3D.

Chapitre 2

Structuration du projet

2.1 Description des besoins

2.1.1 Configuration par fichier externe et parsing

L'un des objectifs fondamentaux de ce projet est de permettre la définition des règles d'un L-système par l'utilisateur sans aucune modification du code source. Bien qu'une interface dédiée soit envisageable, l'utilisation d'un fichier de configuration externe s'avère bien plus pertinente pour automatiser le processus.

Dans cette approche, un simple fichier texte devient un support structuré où l'ensemble des règles et des paramètres d'un L-système est stocké. Pour obtenir l'interprétation d'un modèle spécifique, il suffit de fournir le fichier de configuration correspondant au programme. Cette méthode permet de séparer la logique de calcul de la définition des données, offrant ainsi une grande liberté d'expérimentation.

Pour rendre ce système opérationnel, il est nécessaire de développer un analyseur syntaxique, ou parser, capable de traiter le fichier fourni selon une grammaire prédéfinie. Ce module a pour rôle de lire les instructions textuelles, de vérifier leur validité et de les convertir en objets manipulables par le moteur de calcul.

Le parser doit extraire de manière précise les informations essentielles telles que l'axiome de départ, l'angle de rotation et les règles de production. Ce traitement préalable est indispensable pour transformer une simple suite de caractères en une structure logique prête à être développée par les algorithmes de génération.

2.1.2 Interprétation graphique et moteur de rendu

Le cœur visuel du projet repose sur l'interprétation des règles du L-système via la méthode *Turtle Graphics* [4]. L'enjeu principal ici est transformer une logique récursive en une structure géométrique cohérente, que ce soit dans un plan ou dans l'espace. Pour cela, nous avons développé deux moteurs de rendu spécifiques, pour gèrer l'affichage selon le mode choisi par l'utilisateur.

Visualisation 2D

Pour la visualisation en 2D, le moteur utilise la bibliothèque Java AWT. Ce choix permet de manipuler facilement des lignes et des tracés vectoriels, tout en offrant une gestion fluide des changements d'échelle (zoom) et panning. Le moteur 2D traduit chaque instruction de la `turtle` en un mouvement sur des coordonnées cartésiennes classiques (X, Y) selon un angle de direction `heading` $\vec{H}$ pour produire des formes fractales précises sur le canevas.

Visualisation 3D

Concernant la visualisation en 3D, nous avons intégré la bibliothèque `Java 3D` (J3D) pour franchir la barrière de la profondeur. Contrairement au mode 2D, ce moteur doit gérer un état plus complexe, incluant non seulement la position (X, Y, Z), mais aussi les angles de `heading` $\vec{H}$, de `up` $\vec{U}$ et de `left` $\vec{L}$.

Ces vecteurs forment un repère local orthogonal qui définit précisément l'orientation de la `turtle` dans l'espace. Le vecteur $\vec{H}$ représente la direction vers laquelle la `turtle` avance, tandis que $\vec{U}$ et $\vec{L}$ servent de références pour appliquer les rotations de tangage (*pitch*), de roulis (*roll*) et de lacet (*yaw*) demandées par le système. En convertissant chaque angle du L-système en une transformation de ce repère, le moteur assure un tracé fluide et rigoureux.

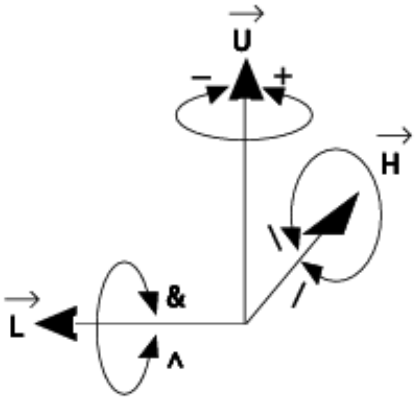

FIGURE 2.1 – Contrôle de la `turtle` en trois dimensions [5]

L'un des avantages majeurs de l'utilisation de `Java 3D` réside dans la gestion automatique des transformations géométriques. Alors que le calcul manuel de l'orientation nécessiterait l'application répétée de matrices de rotation sur les vecteurs $(\vec{H}, \vec{L}, \vec{U})$, J3D nous permet d'implémenter ces changements de repère de manière native et optimisée. Pour les lecteurs souhaitant approfondir les détails mathématiques de ces transformations, le chapitre 1.5 de l'ouvrage *The Algorithmic Beauty of Plants* [5] offre une analyse complète de ces mécanismes.

Enfin, l'utilisation de J3D permet de naviguer librement autour de l'objet généré pour l'observer sous tous ses angles.

2.2 Fonctionnalités implémentées

Dans le cadre de ce projet, nous avons développé et intégré les fonctionnalités suivantes :

— **Parser de configuration :** Nous avons conçu un interpréteur de fichiers de configuration pour charger dynamiquement les axiomes, les règles et les actions sans modifier le code source.
— **Moteur de dérivation polyvalent :** Le système traite les modèles déterministes classiques mais gère également les systèmes **stochastiques**, où le choix des règles dépend de probabilités définies, ainsi que les systèmes **contextuels**, où l'évolution d'un symbole est déterminée par ses voisins directs de gauche et de droite.
— **Double moteur Turtle Graphics :** Une implémentation complète permet de passer de simples fractales en 2D à des modélisations complexes en 3D.
— **Algorithmes de parcours optimisés :** Le rendu utilise un parcours DFS pour l'efficacité mémoire et bascule sur un algorithme BFS pour garantir la cohérence mathématique des systèmes contextuels.

2.3 Organisation du projet

Cette section détaille la répartition des tâches et la gestion du flux de travail, de la conception théorique jusqu'à l'implémentation finale.

— **Arthur LETE :** Concentré sur la logique interne du moteur de derivation (`LSystemEngine`) et le développement des structures de données globales comme les classes `Symbol`, `ActionRule` et `ProductionRule`. Il a assuré la fiabilité du système à travers les phases de débogage et de tests intensifs.
— **Thomas ACAR :** Responsable de l'implémentation complète des fonctions de la `Turtle` (2D et 3D). Il a également conçu l'architecture du moteur de dérivation (`LSystemEngine`) et assuré la planification des structures de données globales comme les classes `Symbol`, `ActionRule` et `ProductionRule` .
— **Zineddine GHALMI :** Développeur du parser `Configuration` pour le traitement des fichiers d'entrée. Il a également conçu une classe de `Logger` personnalisée, facilitant l'interaction avec l'utilisateur et le suivi des flux d'exécution du programme. Par ailleurs, il a travaillé conjointement avec Arthur sur les différentes structures de données et a apporté son soutien aux autres membres de l'équipe sur diverses parties du projet.

Chapitre 3

Éléments techniques

3.1 Description des structures de données

L'architecture du projet repose sur un ensemble de classes qui séparent la représentation des données, les règles de réécriture et l'exécution des commandes graphiques.

3.1.1 Gestion des Symboles via le pattern Flyweight :

— La classe `Symbol` ne possède pas de constructeur public, mais utilise une `Map<Character, Symbol>` statique nommée `instances`.

— Cette structure garantit qu'un caractère (comme 'F' ou '+') correspond à une seule et unique instance en mémoire durant toute l'exécution.

— Cela optimise l'utilisation de la mémoire et centralise les règles : chaque instance unique de `Symbol` stocke sa propre liste de règles de production (`List<ProductionRule>`) et son action associée (`ActionRule`).

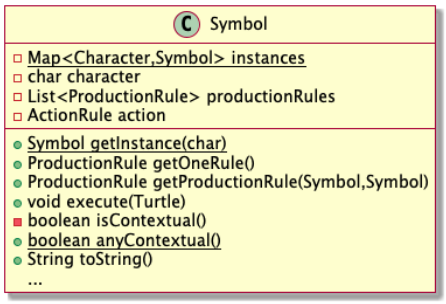

FIGURE 3.1 – Diagramme simplifiée de class Symbol

3.1.2 Encapsulation des Règles de Production :

— La structure de données `ProductionRule` encapsule le résultat d'une réécriture sous la forme d'une `List<Symbol>`, représentant le successeur de la règle.

— Elle intègre également des variables pour gérer des systèmes avancés : un double pour la probabilité d'application (devant respecter la condition $0 \leq p \leq 1$), ainsi que deux caractères (leftContext et rightContext) pour conditionner la règle à son voisinage.

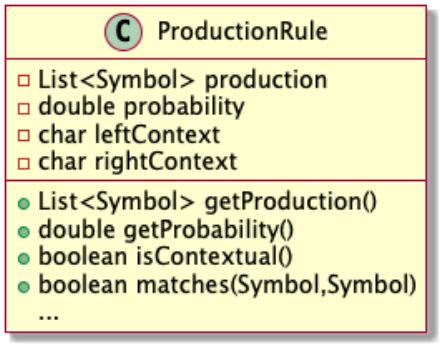

FIGURE 3.2 – Diagramme simplifiée de class ProductionRule

3.1.3 Mappage des Symboles aux Actions :

— Plutôt que d'utiliser de multiples classes pour chaque type de mouvement, la classe ActionRule utilise l'interface fonctionnelle `Consumer<Turtle>` de Java.
— Cette structure permet d'associer une expression lambda (comme `t -> t.turnLeft(`α`)`) à un mot-clé (ex : "left", "move", "push") récupéré lors du parsing du fichier de configuration .
— Grâce à cette approche, chaque symbole peut posséder sa propre `ActionRule` personnalisée. Cela offre un niveau d'abstraction, permettant de dissocier totalement l'aspect graphique du moteur de dérivation.

3.1.4 L'état de `Turtle2D`

Pour modéliser des structures ramifiées (comme des arbres), le système doit être capable de "se souvenir" d'une position et d'une orientation avant de créer une branche, puis d'y revenir une fois la branche terminée. Ce mécanisme repose sur l'utilisation des symboles spéciaux [(push) et] (pop). En interprétation avec J3D, nous utilisons la classe native Transform3D pour résoudre ce problème. Cependant, notre interpréteur 2D étant développé par nos soins à partir du code source, une classe distincte est nécessaire pour effectuer la même tâche.

La classe `State2D` encapsule l'état complet de la `turtle` à un instant t :

— **Position :** Coordonnées cartésiennes (X, Y).
— **Angle :** L'orientation actuelle en degrés.

Dans notre moteur de rendu, nous utilisons une Stack<State2D> pour

— **Symbole [(Push) :** L'état actuel est empilé (sauvegarde).
— **Symbole] (Pop) :** Le dernier état est dépilé et réappliqué à la `turtle` (retour au point de branchement).

3.2 Description des algorithmes

Le moteur `LSystemEngine` et les symboles implémentent plusieurs algorithmes nécéssaires pour générer la fractale, allant de la sélection stochastique à la selection contextuelle.

3.2.1 Algorithme de sélection stochastique (Tirage aléatoire) :

- Pour les L-Systèmes non-contextuel, la méthode getOneRule() de la classe Symbol sélectionne une règle de production au hasard en fonction des probabilités affectées. Il est important de préciser que pour les symboles déterminisites, qui ne possèdent qu'une seule règle de production, cette méthode retournera systématiquement cette unique règle.
- L'algorithme filtre d'abord les règles non contextuelles.
- Ensuite, il génère une valeur aléatoire P_{rand} telle que $0 \leq P_{rand} \leq 1$ à l'aide de Math.random().
- Il itère sur la liste des règles filtrées. Si la valeur de P_{rand} est inférieure ou égale à la probabilité de la règle actuelle (P_i), cette règle est sélectionnée. Sinon, on soustrait P_i de P_{rand} et on passe à la règle suivante.

Algorithme 1 : getOneRule()

Données : Un symbole s, liste de règles $R = \{r_1, \ldots, r_n\}$
Résultat : Une règle de production r_k

$R_f \longleftarrow$ Filtrer règles non contextuelles de s
$P_{rand} \longleftarrow$ `Math.random` ()

pour chaque $r \in R_f$ **faire**
  si $P_{rand} \leq r.prob$ **alors**
    retourner r
  finsi
  $P_{rand} \longleftarrow P_{rand} - r.prob$
fin
retourner r_n

3.2.2 Algorithme de développement récursif (DFS) :

- Pour les systemes non-contextuels, la méthode `draw` et son auxiliaire `drawRecursive` de LSystemEngine effectue un parcours en profondeur (Depth-First) de l'arbre de dérivation du L-Système.
- À chaque appel, si la profondeur cible (depth) atteint 0 ou si le symbole ne possède aucune règle, l'algorithme s'arrête et exécute l'action liée au symbole sur le dessin.
- Sinon, il dérive le symbole actuel et s'appelle récursivement pour chaque Symbol contenu dans la règle de production sélectionnée, en décrémentant la profondeur (depth - 1).

Algorithme 2 : Parcours recursif en profondeur du L-Système

Données : Un symbole s, profondeur actuelle d

```
// Cas de base : profondeur atteinte ou symbole terminal
```
si $d = 0$ ***ou*** *s n'a pas de règle* **alors**
  `exécuterAction` (s)
fin

```
// Cas récursif : dérivation et parcours DFS
```
$r \longleftarrow s$.`getOneRule` ()
foreach *symbole* $s_{child} \in r.getSymbols()$ **do**
  `drawRecursive` $(s_{child}, d-1)$
end

Cette méthode est un pilier de notre projet. Elle permet de traiter les symboles instantanément sans calculer l'intégralité de la dernière génération. Comme les symboles ne sont pas stockés en mémoire (la pile se déchargeant au fur et à mesure), l'empreinte mémoire est drastiquement réduite.

- **Complexité en temps :** $O(n^d)$ (visite de chaque nœud de l'arbre).
- **Complexité en espace :** $O(d)$ (seule la branche actuelle est conservée en mémoire).

où n est le nombre moyen de symboles dans une règle de production. d est la nombre des generations.

En comparaison, la méthode on a vu au section 1.2 aurait une complexité en espace de $O(n^d)$, ce qui devient rapidement impossible à gérer pour des générations élevées, tandis que le DFS reste limité à la profondeur d.

Problématique de l'interprétation des systèmes contextuels

L'utilisation d'un parcours **DFS (Depth-First Search)** pour le rendu immédiat pose un problème majeur de synchronisation temporelle pour les L-Systèmes contextuels. Selon la théorie de **Prusinkiewicz** dans *The Algorithmic Beauty of Plants*, le moteur de dérivation doit respecter le principe de ***parallel rewriting*** : tous les symboles d'une génération n doivent évoluer simultanément vers la génération $n+1$ en se basant sur un état temporel cohérent.

En DFS, un décalage temporel apparaît : au moment d'évaluer un symbole, son voisin de gauche a déjà évolué (génération $n+1$) tandis que son voisin de droite est encore à l'état initial (génération n). Ce manque de simultanéité rend impossible l'application rigoureuse des règles contextuelles ($L < S > R$), car le voisinage immédiat n'appartient pas au même référentiel temporel.

Vers la fin de la phase de développement, nous avions pour objectif d'implémenter les systèmes contextuels. Cependant, l'intégration de ces règles au sein d'un parcours recursif s'est révélée être un défi algorithmique complexe.

Plutôt que de forcer une solution DFS peu fiable, nous avons opté pour une approche **hybride**. Nous avons développé une méthode basée sur le principe du **BFS**, optimisée pour garantir la ***parallel rewriting*** préconisée par **Prusinkiewicz**. Afin de ne pas sacrifier les performances globales, notre moteur de rendu n'active ce mode BFS que si le système

détecte la présence de règles contextuelles. Ainsi, les L-Systèmes classiques continuent de bénéficier de l'optimisation maximale de notre algorithme DFS récursif, tandis que les systèmes complexes profitent d'une dérivation mathématiquement exacte.

3.2.3 Algorithme de développement itératif avec Singletons (BFS) :

- Contrairement aux systèmes classiques, les règles contextuelles nécessitent de connaître l'état de toute la génération au temps t pour calculer la génération t+1.
- La méthode `drawForContext` utilise une `LinkedList<Symbol>` pour stocker l'axiome initial.
- Pour chaque itération (génération), l'algorithme parcourt les symboles en évaluant simultanément leurs voisins de gauche et de droite afin d'identifier une règle contextuelle correspondante.
- Afin de connaitre les index lors de l'évaluation, les futurs remplacements sont isolés dans une `HashMap<Integer, List<Symbol»` où la clé est l'index d'origine.
- Une fois toute la génération évaluée, l'algorithme applique les remplacements en triant les clés par ordre croissant et en calculant un decalage dynamique qui ajuste les index en fonction de la taille de chaque nouvelle chaîne insérée.

Contrairement à la méthode naïve utilisant des `String` (évoquée au section 1.2), cet algorithme optimisé repose sur l'usage de `LinkedList` et du pattern `Flyweight et Singleton` pour la gestion des symboles. Cette approche permet de réduire considérablement la charge computationnelle lors des remplacements ainsi que l'empreinte mémoire, offrant une gestion efficace des itérations.

Chapitre 4

Architecture du projet

4.1 Diagrammes des modules et des classes

Cette section présente l'organisation structurelle de nos classes et modules. Veuillez noter que ces diagrammes privilégient la clarté du contexte architectural et font abstraction des détails techniques secondaires afin de faciliter la compréhension globale du projet.

4.1.1 Structure du paquetage `turtle`

Le paquetage `turtle` repose sur une interface `Turtle` qui unifie les commandes de tracé. Le moteur `Turtle2D` gère le rendu 2D avec une association vers `State2D` pour la gestion de la pile, tandis que `Turtle3D` utilise des transformations matricielles (`Transform3D`). Cette structure permet une transition directe entre les rendus 2D ve 3D. Voir figure 4.1

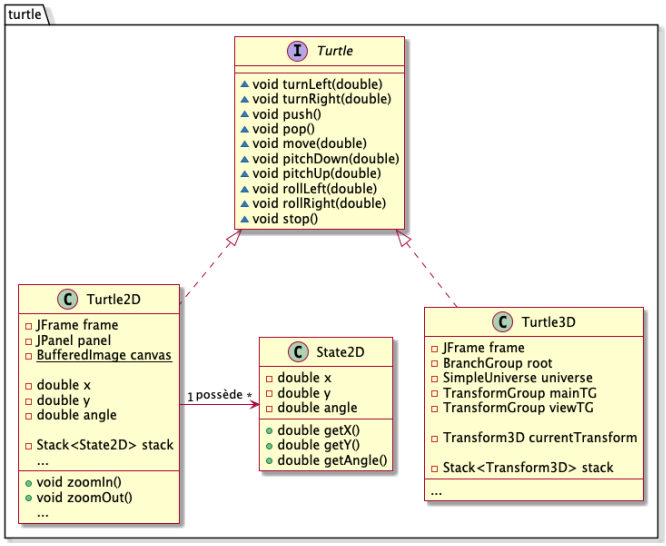

FIGURE 4.1 – Diagramme simplifiée du paquetage `turtle`

4.1.2 Structure du paquetage `fractal`

Ce paquetage constitue le cœur logique du projet, regroupant les algorithmes de dérivation des L-Systèmes, les structures de données associées et la gestion des commandes de tracé. Les détails techniques concernant l'implémentation des structures de données ne sont pas repris ici, car ils font l'objet d'une analyse approfondie dans la section 3.1.

L'architecture repose sur une instanciation unique de chaque `Symbol` par caractère, associant à chacun une action précise et un ensemble de règles de production. Le module `LSystemEngine` exploite ensuite cette structure pour transformer l'axiome initial en une liste de `Symbol` et générer de manière recursive (ou itérative) les générations successives du système. Voir figure 4.2

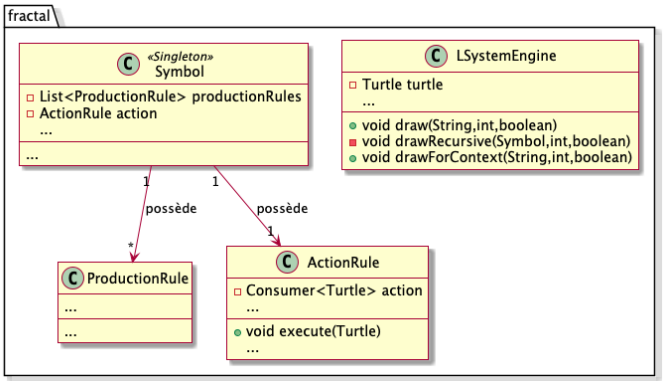

FIGURE 4.2 – Diagramme simplifiée du paquetage `fractal`

4.2 Description des paquetages utilisés

L'application s'appuie sur plusieurs bibliothèques standards et externes pour assurer ses fonctionnalités :

- `java.awt & javax.swing` **:** Utilisés pour la conception de l'interface graphique (JFrame, Panels) et le gestion du rendu 2D sur le canevas.
- `Java 3D (j3dcore, j3dutils)` **:** Ces bibliothèques externes permettent la création et la manipulation de la scène graphique en trois dimensions, incluant la gestion des univers et des transformations spatiales.
- `Vecmath` **:** Utilisé pour les calculs vectoriels et matriciels indispensables aux rotations et aux déplacements de la `turtle` dans l'espace 3D.
- `JogAmp (JOGL)` **:** Fournit les liaisons natives avec OpenGL pour assurer une accélération matérielle performante lors du rendu des structures organiques complexes.

4.3 Chaînes de traitement

Phase d'initialization

Le processus débute par la lecture du fichier de configuration fourni par l'utilisateur, spécifiant également le mode de rendu (2D ou 3D). Un objet `Configuration` analyse ce fichier, valide la syntaxe et extrait l'axiome ainsi que les règles (actions et productions). Une fois

la validation réussie, l'objet `Turtle` approprié selon le mode de rendu est instancié dans `main`. Ensuite, en s'appuyant sur les données extraites par l'objet Configuration le système génère en mémoire les objets `Symbol` uniques, associant chaque caractère à ses comportements respectifs pour préparer la dérivation.

Phase d'évolution et de rendu graphique

Une fois l'initialisation terminée, le moteur `LSystemEngine` prend le contrôle pour générer la structure fractale. Selon la nature des symboles, le système choisit l'algorithme d'évolution optimal : une approche **BFS** (Breadth-First Search) pour les systèmes contextuels ou une approche **DFS** (Depth-First Search) récursive pour les cas standards. Lors de la dernière génération, chaque symbole déclenche son action associée. Les détails algorithmiques de ces processus sont approfondis dans la section 3.2.

L'exécution technique de ces actions repose sur une délégation stricte : le symbole transmet l'instance de la `Turtle` à son `ActionRule`. Cette dernière exécute la commande (ex : *move*, *turn*) sans connaître les détails de l'implémentation graphique. L'aspect visuel est entièrement géré par la `Turtle`, garantissant une séparation claire entre la logique de dérivation et le rendu final. Voir figure 4.3

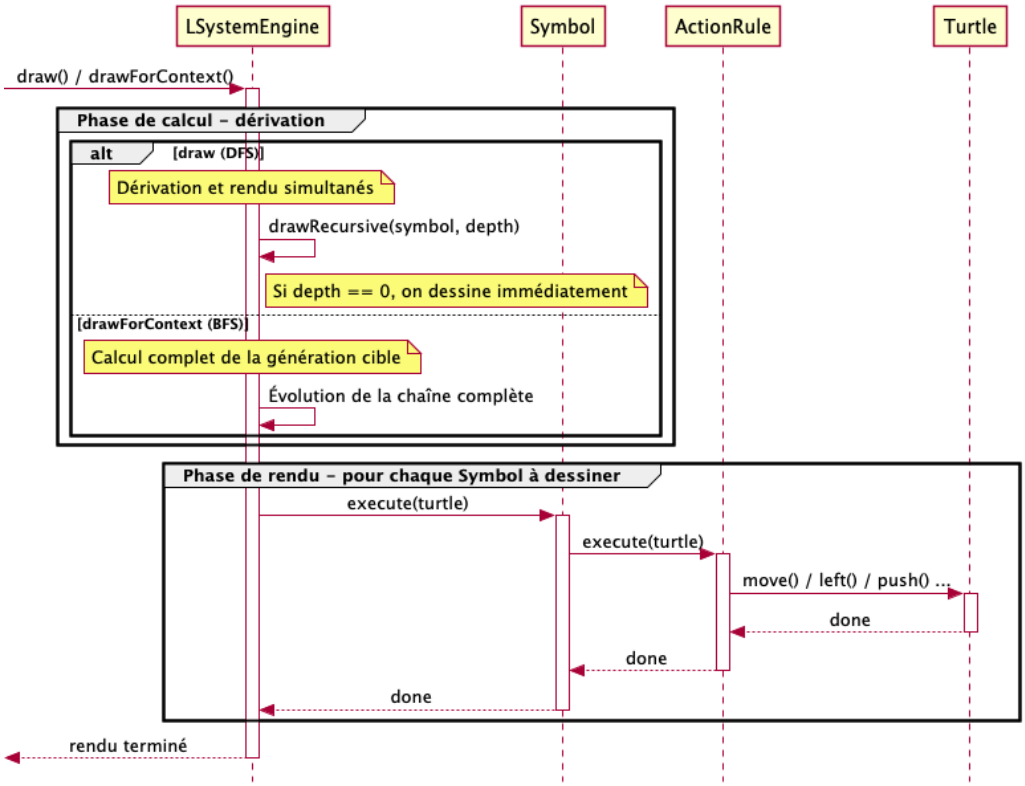

FIGURE 4.3 – Flux d'évolution et de rendu du `LSystemEngine`.

En combinant ces deux phases, nous obtenons la structure globale du programme pilotée par la méthode `main`. Par conséquent, l'organigramme complet du flux de contrôle est illustré dans la figure 4.4.

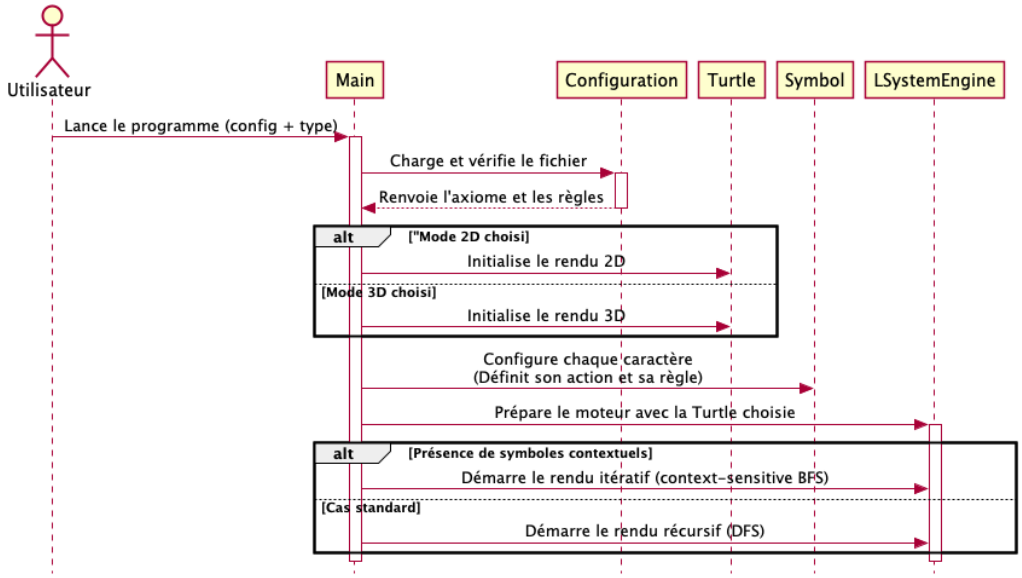

FIGURE 4.4 – Flux de contrôle global.

Chapitre 5

Expérimentations et usages

5.1 Cas d'utilisation

5.1.1 Grammaire du fichier de configuration

Le fichier de configuration est un fichier texte simple permettant de définir entièrement un L-système sans modifier le code source. Il est analysé par la classe `Configuration` qui en extrait toutes les informations nécessaires au moteur de dérivation.

Structure générale

- Chaque directive occupe une seule ligne, de la forme `clé = valeur` ou `clé : valeur`.
- Les **commentaires** sont des lignes commençant par `#` ; elles sont ignorées lors de l'analyse.
- Les **lignes vides** sont ignorées.
- Les espaces autour du séparateur (`=` ou `:`) sont tolérés.
- Les clés sont insensibles à la casse (*case-insensitive*).

Le tableau 5.1 récapitule les directives reconnues par le parser.

Clé	Synonymes	Description
`axiom`	`axiome`	Chaîne de départ du L-système.
`iteration`	—	Nombre de générations à dériver.
`rule`	`regle`, `règle`	Règle de production (réécriture).
`action`	—	Association symbole → commande graphique.

Table 5.1 – Directives reconnues par le fichier de configuration.

Axiome et nombre d'itérations

L'axiome définit la chaîne initiale à partir de laquelle le L-système évolue, et `iteration` fixe la profondeur de dérivation :

```
axiom     = <chaîne>
iteration = <entier>
```

Règles de production

Les règles de production définissent comment un symbole est remplacé à chaque itération. La syntaxe générale est :

```
rule = <spécification_symbole>, <successeur>[:<probabilité>]
```

Trois formes de règles sont supportées :

— **Règle déterministe :** Le symbole `S` est systématiquement remplacé par son successeur.

```
rule = F, F-F++F-F
```

— **Règle stochastique :** La probabilité $p \in [0, 1]$ est ajoutée après le successeur, séparée par `:`. La somme des probabilités de toutes les règles d'un même symbole doit être égale à 1.

```
rule = F, F[+F]F[-F]F:0.33
rule = F, F[+F]F:0.33
rule = F, F[-F]F:0.33
```

— **Règle contextuelle :** Le remplacement n'est appliqué que si les voisins immédiats du symbole correspondent aux contextes `G` (gauche) et `D` (droit). Les deux contextes sont optionnels et utilisables indépendamment.

```
rule = A < B > A, B       (contextes gauche et droit)
rule = F < W, [+L][-R]FW  (contexte gauche seul)
```

Actions graphiques

Les actions associent un symbole à une commande exécutée par la `Turtle`. La syntaxe est :

```
action = <symbole>, <commande>
```

Le tableau 5.2 liste l'ensemble des commandes disponibles.

Commande	Paramètre	Description
`move`	distance	Avance d'une distance donnée en traçant un segment.
`left`	angle (°)	Rotation à gauche d'un angle donné.
`right`	angle (°)	Rotation à droite d'un angle donné.
`up`	angle (°)	Tangage vers le haut (mode 3D uniquement).
`down`	angle (°)	Tangage vers le bas (mode 3D uniquement).
`rollL`	angle (°)	Roulis vers la gauche (mode 3D uniquement).
`rollR`	angle (°)	Roulis vers la droite (mode 3D uniquement).
`push`	—	Sauvegarde l'état courant de la `turtle` sur la pile.
`pop`	—	Restaure le dernier état sauvegardé depuis la pile.

TABLE 5.2 – Commandes d'action disponibles pour la `Turtle`.

Exemples de fichiers de configuration

Courbe de Koch (déterministe, 2D) :

Un exemple minimal illustrant un L-système classique. La règle unique de `F` est déterministe ; les figures 1.1 et 1.2 montrent les générations obtenues.

```
axiom = F
rule = F, F-F++F-F
iteration = 4
action = F, move 50
action = +, left 60
action = -, right 60
```

Fractal Plant (stochastique, 2D) :

Trois règles partiellement probabilistes sont définies pour `F`. À chaque itération, l'une d'elles est choisie aléatoirement, générant ainsi une plante différente à chaque exécution. Les symboles `[` et `]` gèrent les embranchements via un mécanisme de pile (*push/pop*).

```
axiom     = F
rule      = F, F[+F]F[-F]F:0.33
rule      = F, F[+F]F:0.33
rule      = F, F[-F]F:0.33
iteration = 5
action    = F, move 20
action    = +, left 25
action    = -, right 25
action    = [, push
action    = ], pop
```

Propagation d'un signal (contextuel, 2D) :

L'évolution de symbole `B` dépend ici de son voisin gauche.

```
axiom = ABBBB
rule = A<B, A
rule = A, B
...
```

Les actions 3D :

L'ajout des axes de tangage (*pitch up/down*) et de roulis (*roll left/right*) permet au système d'évoluer dans un espace tridimensionnel.

```
...
action = ^, up 90
action = _, down 90
action = /, rollR 90
action = \, rollL 90
...
```

5.1.2 Algorithme de compilation / execution automatisé (ANT)

Etant donné la complexité de notre programme, l'utilisation de diverse bibliothèques, tout en utilisant des fichiers de configurations détaillants des fractales, nous avions besoins d'un outil nous permettant de pouvoir compiler et executer le programme de manière

automatisé, tout en spécifiant certains paramètres. Pour cela nous avons opté pour l'outil ANT qui répond parfaitement à ce problème. En effet nous avons rédigé le script complet s'executant avec la commande suivante :

ant run -Dfile=<filepath> -Dturtle='2d'|'3d' -Dtext='y'|'n'

Cette commande se decompose de la manière suivante :

- 'ant run' -> Compilation complète et execution du programme en ajoutant toutes les bibliothèques externes nécessaires
- '-Dfile=<filepath>' -> Choix du fichier de configuration où <filepath> est le chemin vers le fichier choisit
- '-Dturtle='2d'|'3d' ' -> Choix du type de dessin (2D ou 3D) où '2d'|'3d' est le choix entre '2d' et '3d'
- '-Dtext='y|n' ' -> Choix de sauvegarder la chaine de caractère de la fractale finale dans un fichier, où 'y'|'n' est le choix entre 'yes' et 'no'

5.1.3 Aperçu des résultats visuels

Une fois la configuration établie et l'exécution lancée, le système génère les structures conformément aux paramètres définis. Les figures 5.1, 5.2, 5.3, 5.4 illustrent les résultats obtenus pour différents modèles.

5.2 Résultats quantifiables

Comme nous l'avons mentionné au début de ce rapport, la complexité en temps et en espace attendue pour la **fractale de Koch** ($F \to F - F + +F - F$) suit une croissance exponentielle de type $O(n^d)$, soit ici $O(4^d)$. Nous avons comparé trois approches distinctes : l'algorithme "naïf" qu'on a mentionné dans la section 1.2 optimisé avec **StringBuilder**, notre structure récursive **DFS avec Singleton**, et enfin le système itératif **BFS utilisant une LinkedList des Singletons** pour les besoins contextuels.

L'analyse des données révèle une fracture nette entre la vitesse pure et la gestion de la mémoire, particulièrement flagrante à partir de la 12ème génération :

- **Approche naïve avec StringBuilder (Heap memory) :** Jusqu'à $d = 12$, le StringBuilder est extrêmement performant (∼2,8s). Cependant, sa nature l'oblige à stocker l'intégralité de la chaîne en mémoire vive. À la 14ème génération, la consommation explose à **1,25 GB**. Dès la 15ème génération, le système sature complètement et renvoie une erreur `OutOfMemoryError: Java heap space`. Cette approche est donc limitée physiquement par la RAM disponible.
- **Approche récursive DFS (Stack & Singleton) :** Notre structure récursive déplace le problème vers la pile d'appels (**stack**). Grâce au pattern **Singleton**, nous n'avons que 3 instances de symboles (F, +, -) en mémoire. Alors que le StringBuilder agonise à 1,25 GB, notre DFS ne consomme que **74 KB** pour la même tâche. Nous avons réduit l'empreinte mémoire d'un facteur **20 000**, rendant l'exécution possible là où les autres méthodes échouent.
- **Approche itérative BFS (LinkedList & Singleton) :** L'approche itérative avec LinkedList des pointeurs montre ses limites très tôt. À cause de la gestion massive des

objets, le temps de calcul devient prohibitif dès $d = 8$ (∼5s). Pour $d = 12$, le temps d'exécution théorique dépasse les 35 minutes, ce qui rend cette méthode impraticable pour de hautes itérations.

Génération	**StringBuilder (Heap)**	**Récursive DFS (Stack)**	**Itérative Singleton**
d=5	∼ 3 ms	∼ 10 ms	∼ 12 ms
d=8	∼ 33 ms	∼ 75 ms	∼ 4981 ms
d=12	∼ 2874 ms	∼ 4200 ms	∼ 35-40 min (est.)
d=14	**1,25 GB RAM**	**74 KB RAM**	-
d=15	**OutOfMemoryError**	**Stable & Exécutable**	-

Table 5.3 – Comparaison des performances et de la consommation mémoire.

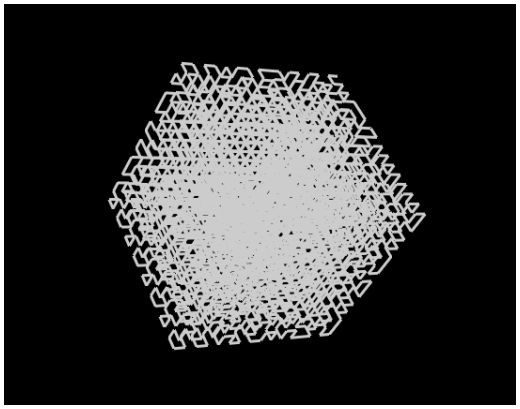

FIGURE 5.1 – 3D visualisation de Hilbert Curve

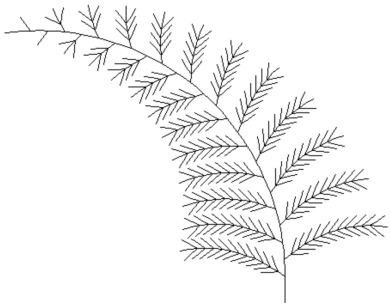

FIGURE 5.2 – 2D visualisation d'un système avec branches.

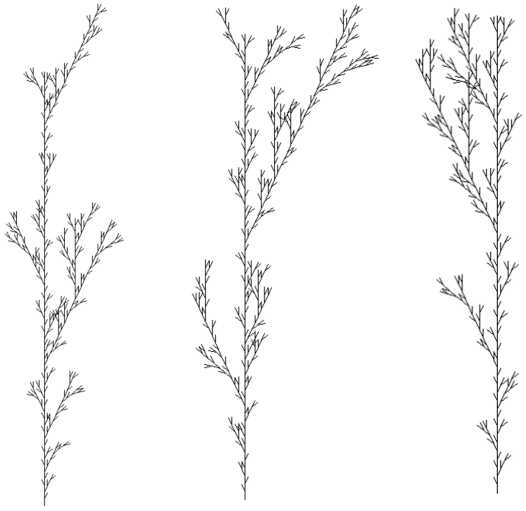

FIGURE 5.3 – Variantes differents d'un système stochastic.

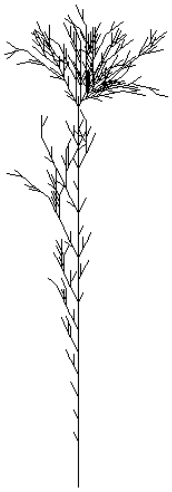

FIGURE 5.4 – 2D visualisation d'un système contextuel.

Chapitre 6

Conclusion

Ce projet nous a permis de concevoir et réaliser un interpréteur de systèmes de Lindenmayer (L-systèmes) en Java, capable de modéliser des structures fractales complexes. L'architecture développée répond avec succès aux objectifs initiaux : grâce à l'analyse d'un fichier de configuration externe, notre système sépare efficacement la définition des données de la logique de calcul. De plus, l'intégration d'un double moteur Turtle Graphics permet une visualisation spatiale fidèle, aussi bien pour des tracés en 2D que pour des modélisations en 3D

D'un point de vue algorithmique, le bilan est particulièrement positif concernant la complexité. L'utilisation d'un parcours récursif (DFS) couplé au pattern Singleton a permis de réduire drastiquement l'empreinte mémoire, rendant l'exécution possible pour un nombre élevé de générations là où une approche naïve échoue. Pour traiter les systèmes contextuels et garantir leur cohérence mathématique, nous avons basculé sur un algorithme itératif basé sur le principe du BFS.

Cependant, nos expérimentations soulignent les limites de cette approche itérative BFS, dont le temps de calcul devient prohibitif à mesure que le nombre de générations augmente en raison de la gestion massive des objets. En perspective d'amélioration, les travaux futurs devraient donc se concentrer sur l'optimisation de cet algorithme itératif ou sur le développement d'un algorithme récursif permettant une écriture parallèle, afin de réduire sa charge computationnelle, permettant ainsi de générer des L-systèmes contextuels de hautes itérations avec une performance fluide.

Bibliographie

[1] EPFL CS108. L-systems (lindenmayer systems). https://cs108.epfl.ch/archive/19/e/LSYS/LSYS.html. Consulté le 11 avril 2026.

[2] Oracle. String (java platform se 8). https://docs.oracle.com/javase/8/docs/api/java/lang/String.html. Consulté le 13 avril 2026.

[3] MIT OpenCourseWare. Software construction - reading 9 : Immutability. https://ocw.mit.edu/ans7870/6/6.005/s16/classes/09-immutability/, 2016. Consulté le 13 avril 2026.

[4] Tao Ju, Quentin Rice, and Michelle Holloway. Turtle geometry in computer graphics and computer aided design. https://www.cs.wustl.edu/~taoju/research/TurtlesforCADRevised.pdf, 2004. Consulté le 13 avril 2026.

[5] Przemyslaw Prusinkiewicz and Aristid Lindenmayer. *The Algorithmic Beauty of Plants*. Springer-Verlag, New York, 1990. Disponible en ligne sur http://algorithmicbotany.org/papers/#abop.